

实验 3 进程通信 I

3.1 实验目的

- (1) 了解系统调用 `pipe()`、`mkfifo()` 的功能;
- (2) 理解 Linux 进程之间通信的基本原理;
- (3) 掌握利用管道机制进行进程间通信的实现方法, 并加深进程通信机制的理解。

3.2 预备知识

在 Linux 系统中, 进程与进程之间需要互相交换数据, 即进程通信。进程通信可以发生在同一主机的进程之间, 也可以发生在不同主机与主机的进程之间。

Linux 下进程间通信的几种主要手段有:

1. 无名管道 (Pipe) 及有名管道 (named pipe): 无名管道可用于具有亲缘关系进程间的通信, 有名管道克服了无名管道没有名字的限制, 因此, 除具有管道所具有的功能外, 它还允许无亲缘关系进程间的通信;

2. 信号 (Signal): 信号是比较复杂的通信方式, 用于通知接受进程有某种事件发生, 除了用于进程间通信外, 进程还可以发送信号给进程本身; Linux 除了支持 Unix 早期信号语义函数 `sigal` 外, 还支持语义符合 Posix.1 标准的信号函数 `sigaction` (实际上, 该函数是基于 BSD 的, BSD 为了实现可靠信号机制, 又能够统一对外接口, 用 `sigaction` 函数重新实现了 `signal` 函数);

3. 报文 (Message) 队列 (消息队列): 消息队列是消息的链接表, 包括 Posix 消息队列 systemV 消息队列。有足够权限的进程可以向队列中添加消息, 被赋予读权限的进程则可以读走队列中的消息。消息队列克服了信号承载信息量少, 管道只能承载无格式字节流以及缓冲区大小受限等缺点。

4. 共享内存: 使得多个进程可以访问同一块内存空间, 是最快的可用 IPC 形式。是针对其他通信机制运行效率较低而设计的。往往与其它通信机制, 如信号量结合使用, 来达到进程间的同步及互斥。

5. 信号量 (semaphore): 主要作为进程间以及同一进程不同线程之间的同步手段。

6. 套接口 (Socket): 更为一般的进程间通信机制, 可用于不同机器之间的进程间通信。起初是由 Unix 系统的 BSD 分支开发出来的, 但现在一般可以移植到其它类 Unix 系统上: Linux 和 System V 的变种都支持套接字。

本次实验只讨论管道通信方式:

管道允许进程之间按照先进先出的方式传送数据。管道的实现是采用一个连接写进程和读进程的 filesystem 中的共享文件作为数据存储, 所以该管道又称为管道 (pipe) 文件。写进程将数据写入管道文件, 读进程从管道文件中读出数据。管道是一种临界资源, 所以对管道的应用要考虑进程的同步。

3.3 实验内容

1. 实现无名管理单进程的通信；
2. 编写一段程序，使其用管道来实现父子进程之间的进程通信。子进程向父进程发送自己的进程标识符，以及字符串“is sending a message to parent！”。父进程则通过管道读出子进程发来的消息，将消息显示在屏幕上，然后终止。
3. 实现有名管道通信及有名管道的删除。

3.4 实验指导

1.创建无名管道的系统调用格式为：

```
int pipe(filedes);  
int filedes[2];
```

其中 filedes 是文件描述符，由 filedes[0]和 filedes[1]组成。filedes[1]是写入管道的描述符，filedes[0]是从管道读的描述符。

2.进程写管道

进程向管道写数据时必须注意管道的大小限制，管道中已有的数据与要写入之和不能大于管道的容量。

写管道的系统调用格式为：

```
write(filedes[1], buf, size);  
int filedes[1], size;  
char buf[];
```

其中 filedes[1]为写入管道的描述符；buf 是要写入管道的数据；size 是写入的数据长度。

3.进程读管道

进程从管道中读数据时，如果管道中已经有数据，进程按照先进先出的原则通过读指针读取数据。

读管道的系统调用格式为：

```
read(filedes[0], buf, size);  
int filedes[0], size;  
char buf[];
```

其中 filedes[0]为从管道读的描述符；从管道中读出的数据放入 buf 中；size 是读出数据的长度。

4.有名管道

无名管道不足之处是没有名字，因此，只能用于具有亲缘关系的进程间通信。在有名管道（named pipe 或 FIFO）提出来之后，该限制得到了克服。FIFO 不同于管道之处在于它提供一个路径与之关联，以 FIFO 的文件形式存储于文件系统中。有名管道是一个设备文件。因此，即使进程与创建 FIFO 的进程不存在亲缘关系，只要可以访问该路径，就能够通过 FIFO 相互通信。值得注意的是，FIFO（first in first out）总是按照先进先出的原则工作，第一个被写入的数据将首先从管道中读出。

创建有名管道的函数有两个：mknod 和 mkfifo，此处仅以 mkfifo 为例

```
int mkfifo(const char *pathname, mode_t mode);
```

有名管道创建后就可以使用了，有名管道和管道的使用方法基本是相同的。只是使用有名管道时，必须先调用 `open` 将其打开。因为有名管道是一个存在于硬盘上的文件，而管道是存在于内存中的特殊文件。

需要注意的是，调用 `open` 打开有名管道的进程可能会被阻塞。但如果同时用读写方式（`O_RDWR`）打开，则一定不会导致阻塞；如果以只读方式（`O_RDONLY`）打开，则调用 `open` 函数的进程将会被阻塞直到有写方式打开管道；同样以写方式（`O_WRONLY`）打开也会阻塞直到有读方式打开管道。

3.5 参考源代码

1. 无名管道读写字符串（单进程）

```
//3-1.c

#include<stdio.h>
#include<string.h>
#include<unistd.h>
#include<fcntl.h>
char string[]="Welcome to unnamed pipe";
int main(){
    int filed[2];
    char inbuf[256];
    if(pipe(filed)<0){    //创建管道
        printf("Create unnamed pipe failed\n");
        return 1;
    }
    write(filed[1],string,23); //写入管道
    memset(inbuf,0,sizeof(inbuf)); // 把缓冲区内容都置为 0
    read(filed[0],inbuf,23); //读取管道内容到缓冲区
    printf("%s\n",inbuf);    //打印缓冲区信息
    return 0;
}
```

2.父子进程通过管道进行通信

```
//父子进程通信
#include<stdio.h>
#include<stdlib.h>
#include<sys/types.h>
#include<sys/stat.h>
#include<fcntl.h>
#include<unistd.h>
int main(){
    int pid1,pid2;
    int fd[2];
    char out[512],in[512];
    pipe(fd);
    pid1=fork();
    if(pid1==0){
        sprintf(out,"%d is sending a message to parent!\n",getpid());
        printf("%s",out);
        write(fd[1],out,sizeof(out));
    }else{
        read(fd[0],in,sizeof(out));
        printf("%s",in);
    }
    return 0;
}
```

3.有名管道

```
1 //写管道
2 #include<unistd.h>
3 #include<sys/types.h>
4 #include<sys/stat.h>
5 #include<errno.h>
6 #include<fcntl.h>
7 #include<stdlib.h>
8 #include<stdio.h>
9 #include<limits.h>
10 #define MYFIFO "/home/lgx/kk/myfifo"
11 #define MAX_BUFFER_SIZE PIPE_BUF
12
13 int main(int argc,char *argv[])
14 {
15     int fd;
16     char buff[MAX_BUFFER_SIZE];
17     int nwrite;
18 }
```

```

19     if (argc<=1)
20     {
21         printf("Usage: ./fifo_write string\n");
22         exit(1);
23     }
24     sscanf(argv[1], "%s", buff);
25     fd=open(MYFIFO, O_WRONLY);
26     if (fd==-1)
27     {
28         printf("Open fifo file error\n");
29         exit(2);
30     }
31     if ((nwrite=write(fd, buff, MAX_BUFFER_SIZE))>0)
32     {
33         printf("write '%s' to FIFO\n", buff);
34     }
35     close(fd);
36     exit(0);
37 }

```

```

1 //读管道
2 #include<unistd.h>
3 #include<string.h>
4 #include<sys/types.h>
5 #include<sys/stat.h>
6 #include<errno.h>
7 #include<fcntl.h>
8 #include<stdlib.h>
9 #include<stdio.h>
10 #include<limits.h>
11 #define MYFIFO "/home/lgx/kk/myfifo"
12 #define MAX_BUFFER_SIZE PIPE_BUF
13 int main(int argc, char *argv[])
14 {
15     int fd;
16     char buff[MAX_BUFFER_SIZE];
17     int nread;
18
19     if (access(MYFIFO, F_OK)==-1)
20     {
21         if ((mkfifo(MYFIFO, 0666)<0) && (errno!=EEXIST))
22         {
23             printf("Cannot create fifo file\n");
24             exit(1);
25         }
26     }
27     fd=open(MYFIFO, O_RDONLY);
28     if (fd==-1)
29     {
30         printf("Open fifo file error\n");
31         exit(2);
32     }
33     while(1)
34     {
35         memset(buff, 0, sizeof(buff));
36         if ((nread=read(fd, buff, MAX_BUFFER_SIZE))>0)
37         {
38             printf("Read '%s' from FIFO\n", buff);
39         }
40     }
41     close(fd);
42     exit(0);
43 }

```